sesa lab
SOFTWARE ENGINEERING
SALERNO

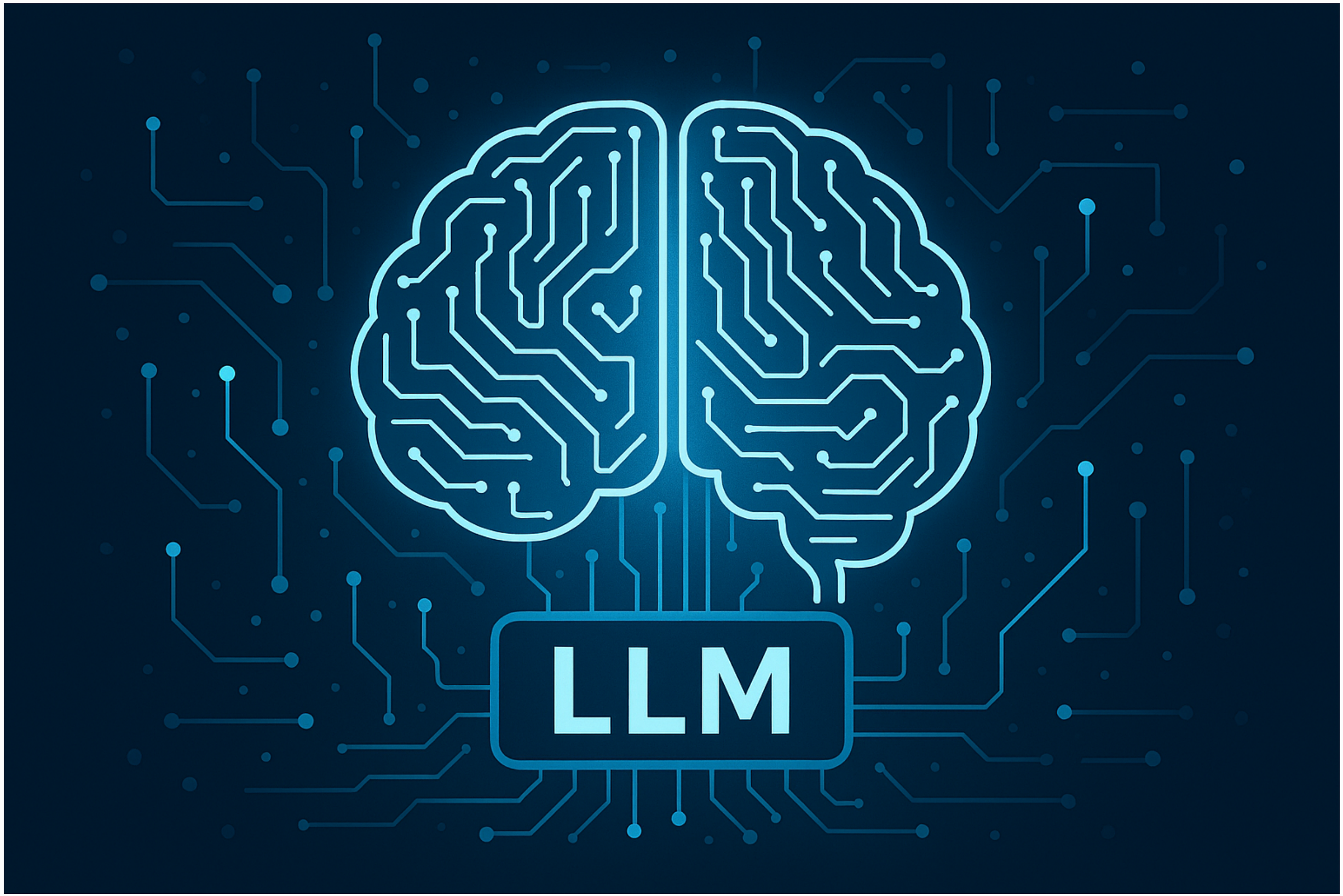
sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

Sustainable Software Engineering: Energy Metrics and the Measurement Procedure



Slides courtesy of Dario Di Nucci | 2026 - 2027

Generative AI is eating the world!



Bitcoins (BTC) are eating the world!

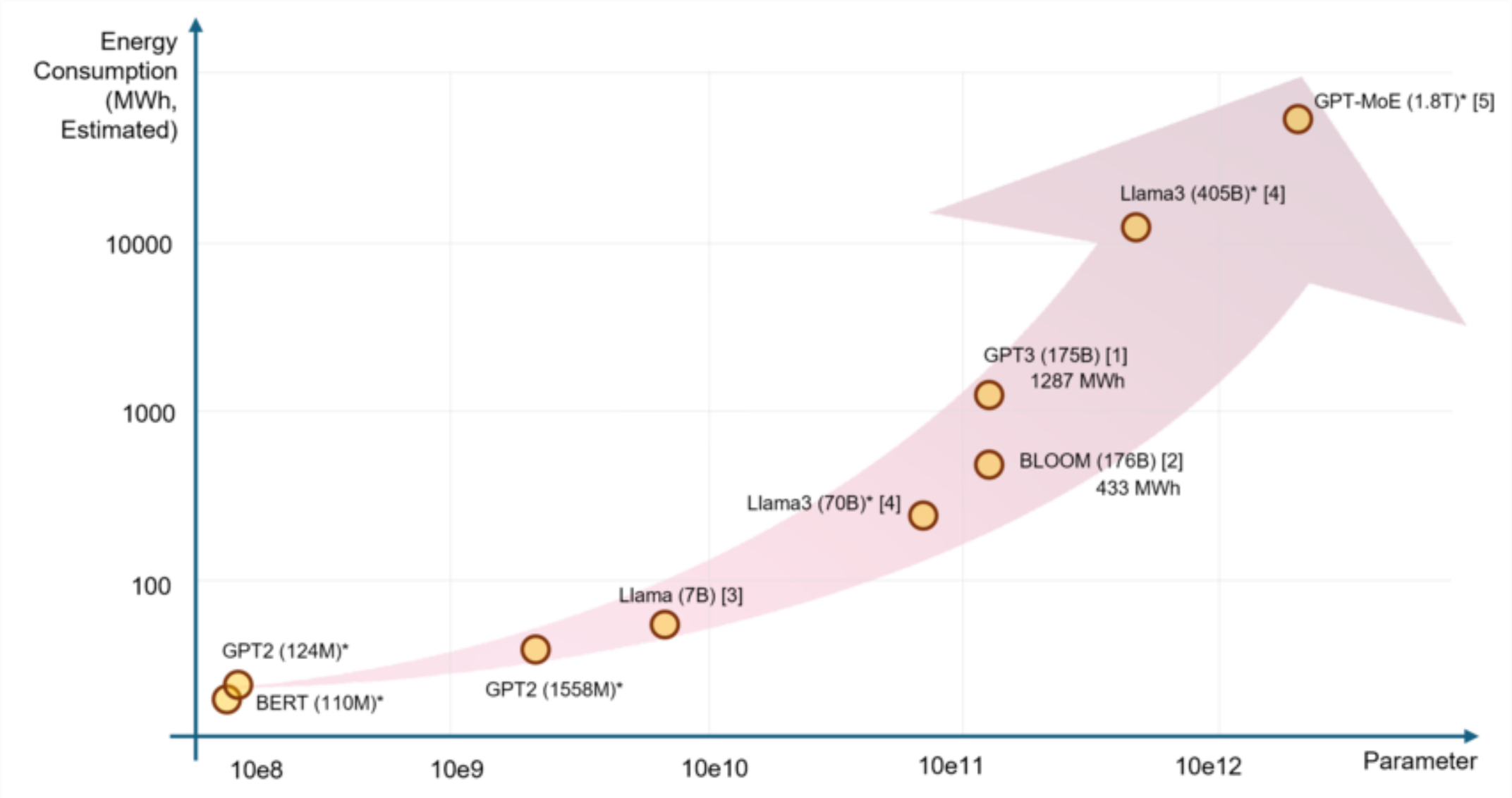


What do Generative AI and Bitcoins have in common?





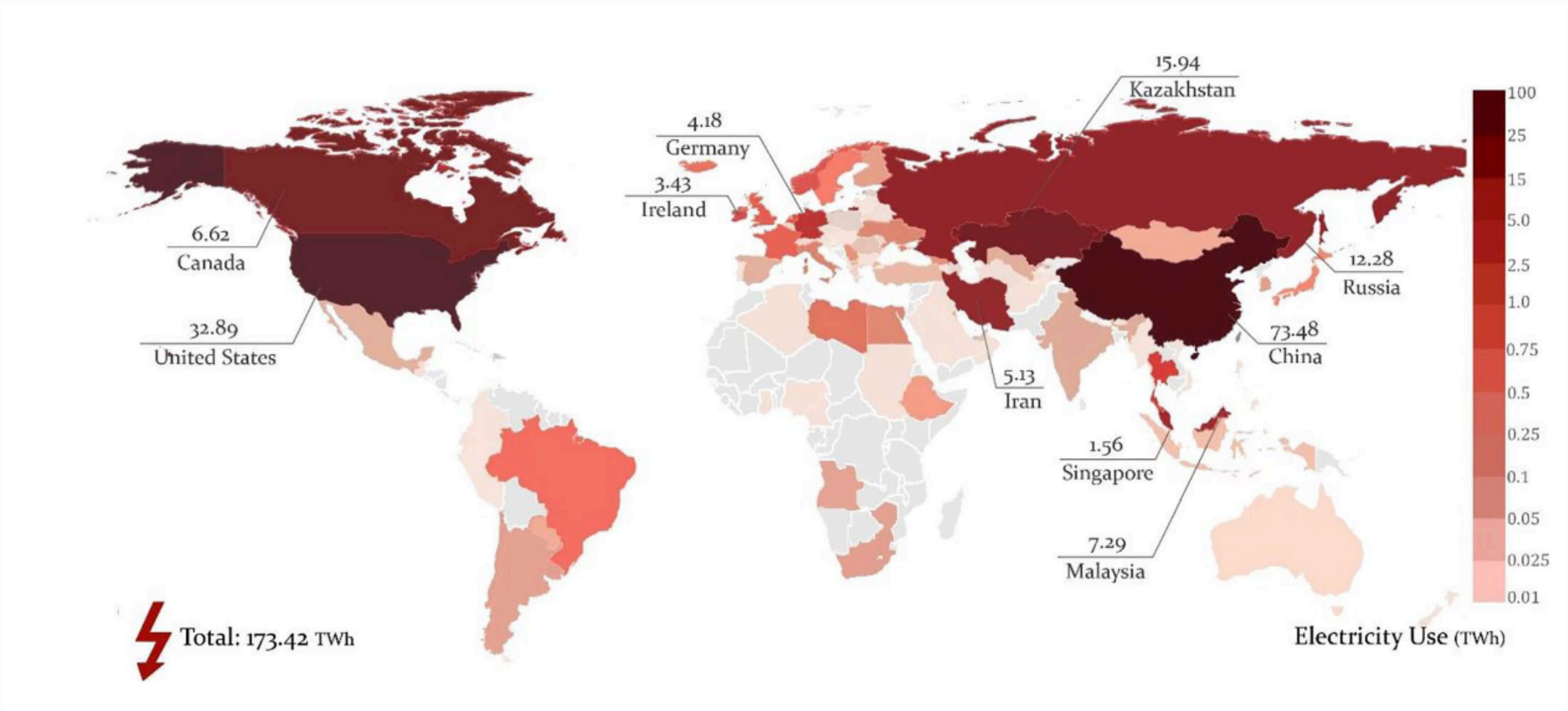
What do Generative AI and Bitcoins have in common?



Energy consumption of training different LLM models with respect to model parameters

https://doi.org/10.48550/arXiv.2409.11416

What do Generative AI and Bitcoins have in common?

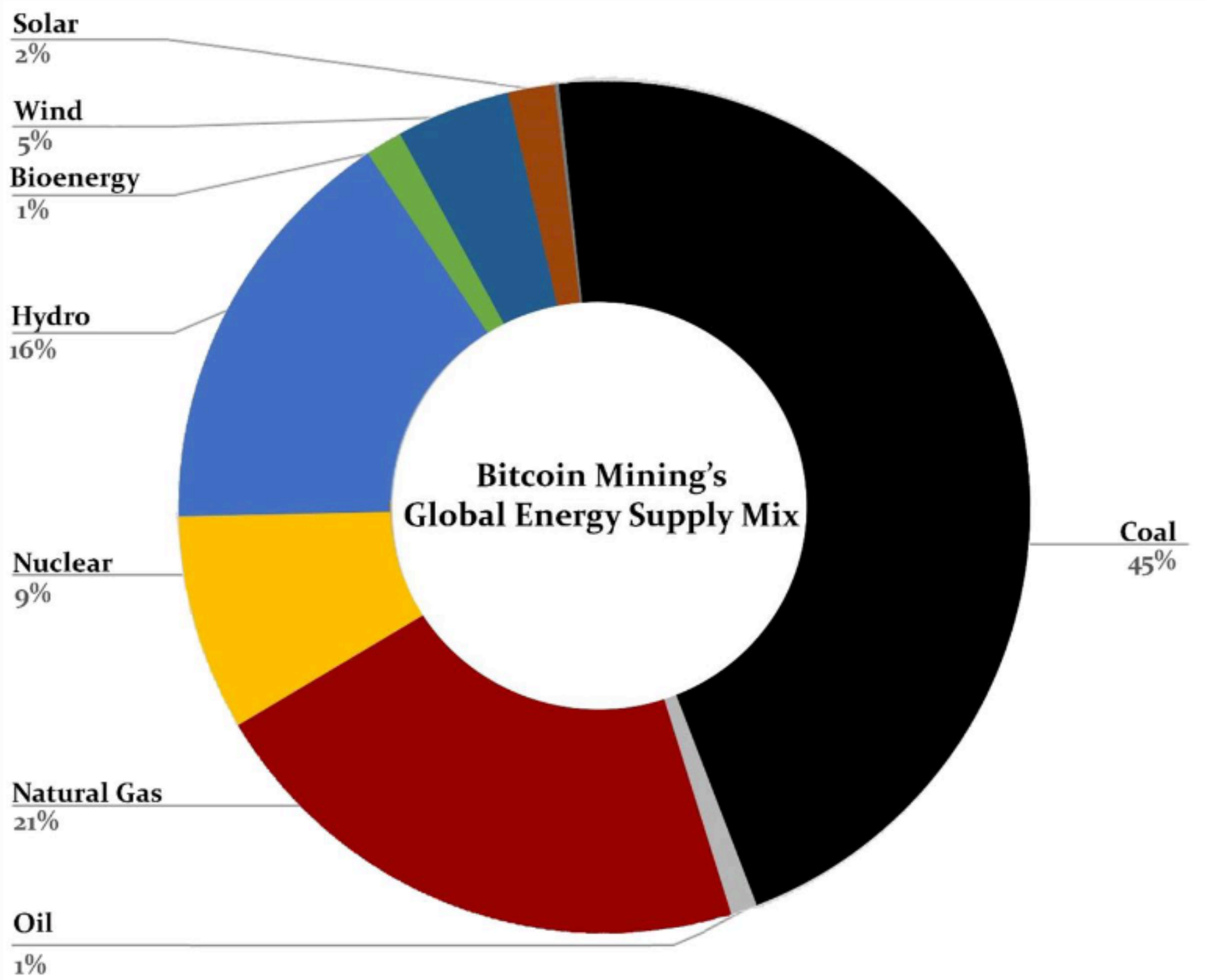


Electricity use of BTC mining across the world (2020-2021)

https://doi.org/10.1029/2023EF003871

https://ccaf.io/cbnsi/cbeci

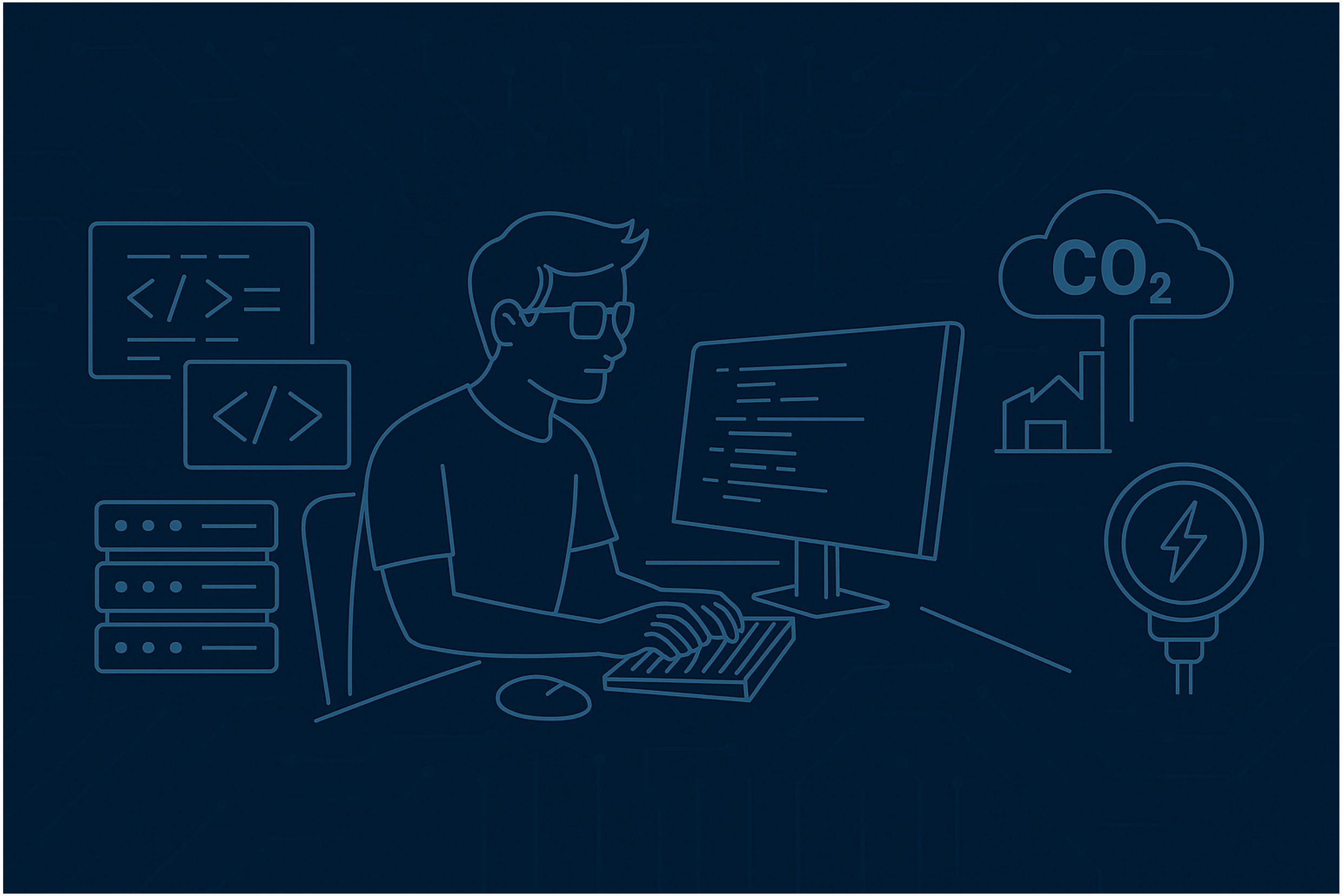
What do Generative AI and Bitcoins have in common?



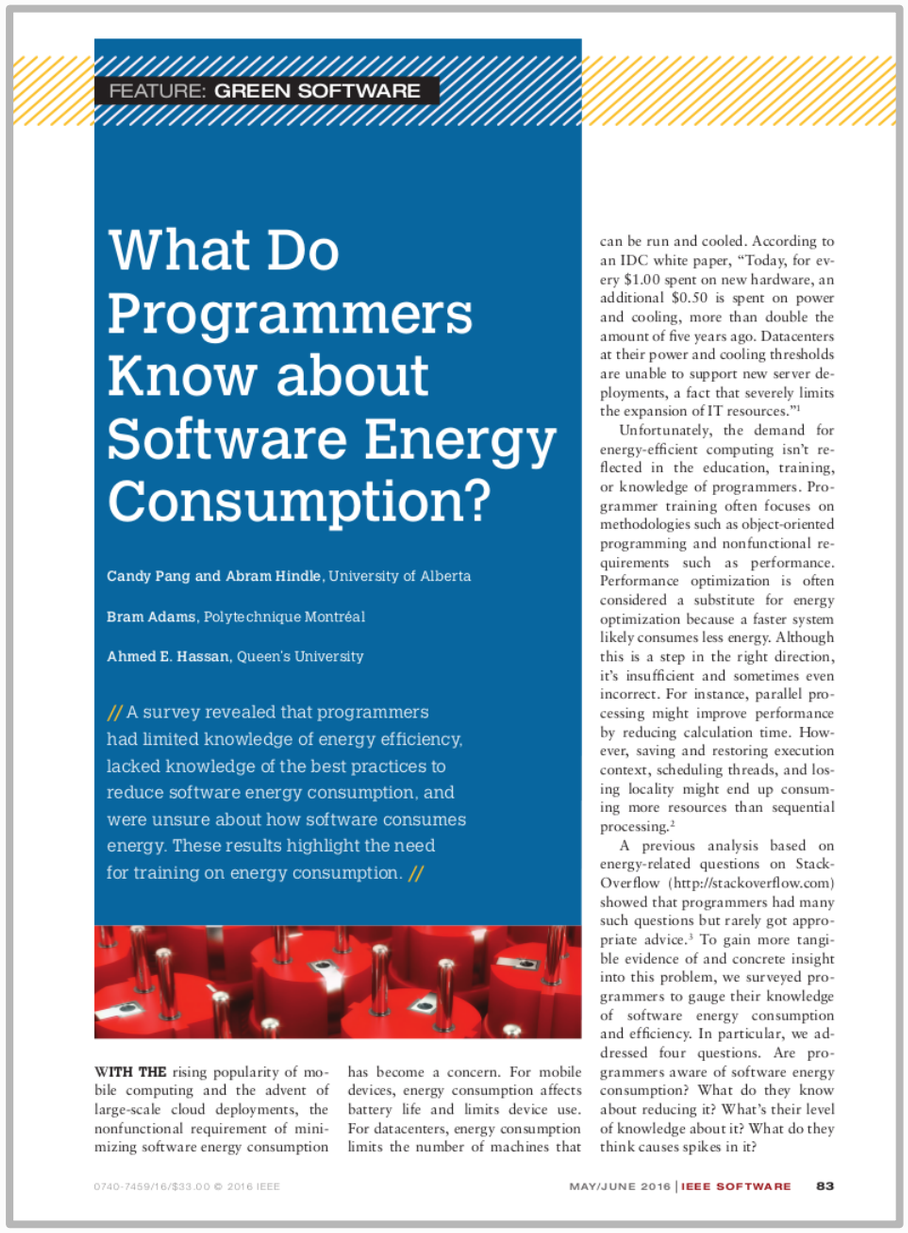
Contributions of energy sources to the global BTC mining network (2020–2021)

https://doi.org/10.1029/2023EF003871

Developers and Energy Consumption



Developers and Energy Consumption



FEATURE: GREEN SOFTWARE

What Do Programmers Know about Software Energy Consumption?

Candy Pang and Abram Hindle, University of Alberta

Bram Adams, Polytechnique Montréal

Ahmed E. Hassan, Queen's University

// A survey revealed that programmers had limited knowledge of energy efficiency, lacked knowledge of the best practices to reduce software energy consumption, and were unsure about how software consumes energy. These results highlight the need for training on energy consumption. //

WITH THE rising popularity of mobile computing and the advent of large-scale cloud deployments, the nonfunctional requirement of minimizing software energy consumption has become a concern. For mobile devices, energy consumption affects battery life and limits device use. For datacenters, energy consumption limits the number of machines that can be run and cooled. According to an IDC white paper, "Today, for every $1.00 spent on new hardware, an additional $0.50 is spent on power and cooling, more than double the amount of five years ago. Datacenters at their power and cooling thresholds are unable to support new server deployments, a fact that severely limits the expansion of IT resources."[1]

Unfortunately, the demand for energy-efficient computing isn't reflected in the education, training, or knowledge of programmers. Programmer training often focuses on methodologies such as object-oriented programming and nonfunctional requirements such as performance. Performance optimization is often considered a substitute for energy optimization because a faster system likely consumes less energy. Although this is a step in the right direction, it's insufficient and sometimes even incorrect. For instance, parallel processing might improve performance by reducing calculation time. However, saving and restoring execution context, scheduling threads, and losing locality might end up consuming more resources than sequential processing.[2]

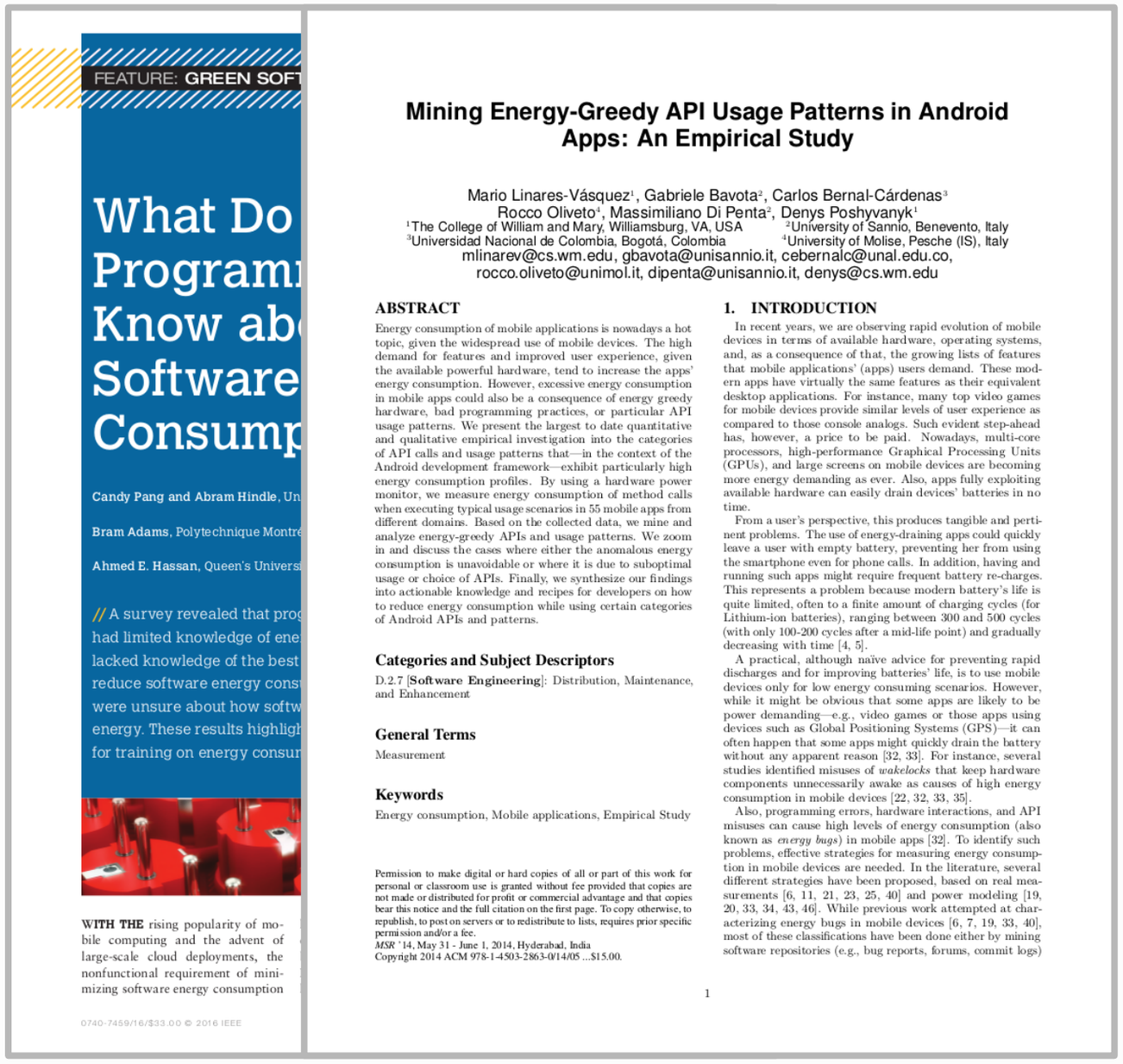
A previous analysis based on energy-related questions on StackOverflow (http://stackoverflow.com) showed that programmers had many such questions but rarely got appropriate advice.[3] To gain more tangible evidence of and concrete insight into this problem, we surveyed programmers to gauge their knowledge of software energy consumption and efficiency. In particular, we addressed four questions. Are programmers aware of software energy consumption? What do they know about reducing it? What's their level of knowledge about it? What do they think causes spikes in it?

0740-7459/16/$33.00 © 2016 IEEE

Programmers often have limited knowledge of energy efficiency, are unaware of best practices for reducing software energy consumption, and are uncertain about how software utilizes energy.

https://ieeexplore.ieee.org/abstract/document/7155416

Developers and Energy Consumption



FEATURE: GREEN SOFT

What Do Programi Know ab Software Consump

Candy Pang and Abram Hindle, Un

Bram Adams, Polytechnique Montre

Ahmed E. Hassan, Queen's Universi

// A survey revealed that prog had limited knowledge of ene lacked knowledge of the best reduce software energy cons were unsure about how softw energy. These results highligh for training on energy consur

WITH THE rising popularity of mobile computing and the advent of large-scale cloud deployments, the nonfunctional requirement of minimizing software energy consumption

0740-7459/16/$33.00 © 2016 IEEE

Mining Energy-Greedy API Usage Patterns in Android Apps: An Empirical Study

Mario Linares-Vásquez[1], Gabriele Bavota[2], Carlos Bernal-Cárdenas[3]
Rocco Oliveto[4], Massimiliano Di Penta[2], Denys Poshyvanyk[1]
[1]The College of William and Mary, Williamsburg, VA, USA [2]University of Sannio, Benevento, Italy
[3]Universidad Nacional de Colombia, Bogotá, Colombia [4]University of Molise, Pesche (IS), Italy
mlinarev@cs.wm.edu, gbavota@unisannio.it, cebernalc@unal.edu.co,
rocco.oliveto@unimol.it, dipenta@unisannio.it, denys@cs.wm.edu

ABSTRACT

Energy consumption of mobile applications is nowadays a hot topic, given the widespread use of mobile devices. The high demand for features and improved user experience, given the available powerful hardware, tend to increase the apps' energy consumption. However, excessive energy consumption in mobile apps could also be a consequence of energy greedy hardware, bad programming practices, or particular API usage patterns. We present the largest to date quantitative and qualitative empirical investigation into the categories of API calls and usage patterns that—in the context of the Android development framework—exhibit particularly high energy consumption profiles. By using a hardware power monitor, we measure energy consumption of method calls when executing typical usage scenarios in 55 mobile apps from different domains. Based on the collected data, we mine and analyze energy-greedy APIs and usage patterns. We zoom in and discuss the cases where either the anomalous energy consumption is unavoidable or where it is due to suboptimal usage or choice of APIs. Finally, we synthesize our findings into actionable knowledge and recipes for developers on how to reduce energy consumption while using certain categories of Android APIs and patterns.

Categories and Subject Descriptors

D.2.7 [**Software Engineering**]: Distribution, Maintenance, and Enhancement

General Terms

Measurement

Keywords

Energy consumption, Mobile applications, Empirical Study

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
MSR '14, May 31 - June 1, 2014, Hyderabad, India
Copyright 2014 ACM 978-1-4503-2863-0/14/05 ...$15.00.

1. INTRODUCTION

In recent years, we are observing rapid evolution of mobile devices in terms of available hardware, operating systems, and, as a consequence of that, the growing lists of features that mobile applications' (apps) users demand. These modern apps have virtually the same features as their equivalent desktop applications. For instance, many top video games for mobile devices provide similar levels of user experience as compared to those console analogs. Such evident step-ahead has, however, a price to be paid. Nowadays, multi-core processors, high-performance Graphical Processing Units (GPUs), and large screens on mobile devices are becoming more energy demanding as ever. Also, apps fully exploiting available hardware can easily drain devices' batteries in no time.

From a user's perspective, this produces tangible and pertinent problems. The use of energy-draining apps could quickly leave a user with empty battery, preventing her from using the smartphone even for phone calls. In addition, having and running such apps might require frequent battery re-charges. This represents a problem because modern battery's life is quite limited, often to a finite amount of charging cycles (for Lithium-ion batteries), ranging between 300 and 500 cycles (with only 100-200 cycles after a mid-life point) and gradually decreasing with time [4, 5].

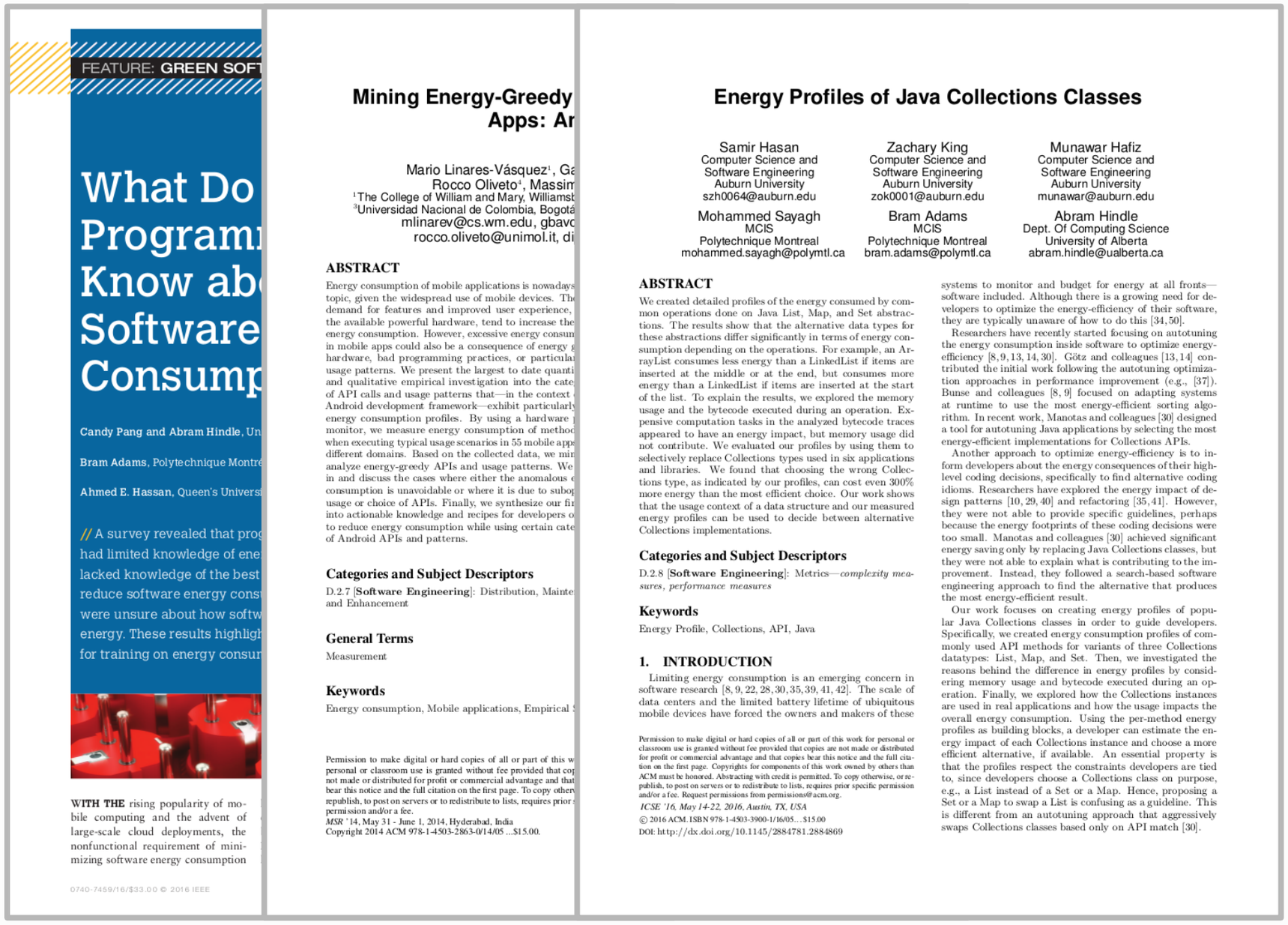
A practical, although naïve advice for preventing rapid discharges and for improving batteries' life, is to use mobile devices only for low energy consuming scenarios. However, while it might be obvious that some apps are likely to be power demanding—e.g., video games or those apps using devices such as Global Positioning Systems (GPS)—it can often happen that some apps might quickly drain the battery without any apparent reason [32, 33]. For instance, several studies identified misuses of *wakelocks* that keep hardware components unnecessarily awake as causes of high energy consumption in mobile devices [22, 32, 33, 35].

Also, programming errors, hardware interactions, and API misuses can cause high levels of energy consumption (also known as *energy bugs*) in mobile apps [32]. To identify such problems, effective strategies for measuring energy consumption in mobile devices are needed. In the literature, several different strategies have been proposed, based on real measurements [6, 11, 21, 23, 25, 40] and power modeling [19, 20, 33, 34, 43, 46]. While previous work attempted at characterizing energy bugs in mobile devices [6, 7, 19, 33, 40], most of these classifications have been done either by mining software repositories (e.g., bug reports, forums, commit logs)

The most comprehensive empirical study of API calls and usage patterns revealed that Android APIs have exceptionally high energy consumption profiles.

https://dl.acm.org/doi/pdf/10.1145/2597073.2597085

Developers and Energy Consumption



FEATURE: GREEN SOFT

What Do Programi Know ab Software Consump

Candy Pang and Abram Hindle, Un

Bram Adams, Polytechnique Montre

Ahmed E. Hassan, Queen's Universi

// A survey revealed that prog had limited knowledge of ene lacked knowledge of the best reduce software energy cons were unsure about how softw energy. These results highligh for training on energy consur

WITH THE rising popularity of mobile computing and the advent of large-scale cloud deployments, the nonfunctional requirement of minimizing software energy consumption

0740-7459/16/$33.00 © 2016 IEEE

Mining Energy-Greedy Apps: Ar

Mario Linares-Vásquez[1], Ga Rocco Oliveto[4], Massim
[1]The College of William and Mary, Williamsb
[3]Universidad Nacional de Colombia, Bogotá
mlinarev@cs.wm.edu, gbavc rocco.oliveto@unimol.it, di

ABSTRACT

Energy consumption of mobile applications is nowadays topic, given the widespread use of mobile devices. The demand for features and improved user experience, the available powerful hardware, tend to increase the energy consumption. However, excessive energy consum in mobile apps could also be a consequence of energy g hardware, bad programming practices, or particular usage patterns. We present the largest to date quanti and qualitative empirical investigation into the cate of API calls and usage patterns that—in the context Android development framework—exhibit particularly energy consumption profiles. By using a hardware monitor, we measure energy consumption of method when executing typical usage scenarios in 55 mobile apps different domains. Based on the collected data, we min analyze energy-greedy APIs and usage patterns. We in and discuss the cases where either the anomalous consumption is unavoidable or where it is due to subop usage or choice of APIs. Finally, we synthesize our fi into actionable knowledge and recipes for developers o to reduce energy consumption while using certain cate of Android APIs and patterns.

Categories and Subject Descriptors

D.2.7 [Software Engineering]: Distribution, Mainte and Enhancement

General Terms

Measurement

Keywords

Energy consumption, Mobile applications, Empirical

Permission to make digital or hard copies of all or part of this w personal or classroom use is granted without fee provided that co not made or distributed for profit or commercial advantage and that bear this notice and the full citation on the first page. To copy other republish, to post on servers or to redistribute to lists, requires prior permission and/or a fee.
MSR '14, May 31 - June 1, 2014, Hyderabad, India
Copyright 2014 ACM 978-1-4503-2863-0/14/05 ...$15.00.

Energy Profiles of Java Collections Classes

Samir Hasan
Computer Science and Software Engineering
Auburn University
szh0064@auburn.edu

Zachary King
Computer Science and Software Engineering
Auburn University
zok0001@auburn.edu

Munawar Hafiz
Computer Science and Software Engineering
Auburn University
munawar@auburn.edu

Mohammed Sayagh
MCIS
Polytechnique Montreal
mohammed.sayagh@polymtl.ca

Bram Adams
MCIS
Polytechnique Montreal
bram.adams@polymtl.ca

Abram Hindle
Dept. Of Computing Science
University of Alberta
abram.hindle@ualberta.ca

ABSTRACT

We created detailed profiles of the energy consumed by common operations done on Java List, Map, and Set abstractions. The results show that the alternative data types for these abstractions differ significantly in terms of energy consumption depending on the operations. For example, an ArrayList consumes less energy than a LinkedList if items are inserted at the middle or at the end, but consumes more energy than a LinkedList if items are inserted at the start of the list. To explain the results, we explored the memory usage and the bytecode executed during an operation. Expensive computation tasks in the analyzed bytecode traces appeared to have an energy impact, but memory usage did not contribute. We evaluated our profiles by using them to selectively replace Collections types used in six applications and libraries. We found that choosing the wrong Collections type, as indicated by our profiles, can cost even 300% more energy than the most efficient choice. Our work shows that the usage context of a data structure and our measured energy profiles can be used to decide between alternative Collections implementations.

Categories and Subject Descriptors

D.2.8 [Software Engineering]: Metrics—*complexity measures, performance measures*

Keywords

Energy Profile, Collections, API, Java

1. INTRODUCTION

Limiting energy consumption is an emerging concern in software research [8, 9, 22, 28, 30, 35, 39, 41, 42]. The scale of data centers and the limited battery lifetime of ubiquitous mobile devices have forced the owners and makers of these systems to monitor and budget for energy at all fronts—software included. Although there is a growing need for developers to optimize the energy-efficiency of their software, they are typically unaware of how to do this [34, 50].

Researchers have recently started focusing on autotuning the energy consumption inside software to optimize energy-efficiency [8, 9, 13, 14, 30]. Götz and colleagues [13, 14] contributed the initial work following the autotuning optimization approaches in performance improvement (e.g., [37]). Bunse and colleagues [8, 9] focused on adapting systems at runtime to use the most energy-efficient sorting algorithm. In recent work, Manotas and colleagues [30] designed a tool for autotuning Java applications by selecting the most energy-efficient implementations for Collections APIs.

Another approach to optimize energy-efficiency is to inform developers about the energy consequences of their high-level coding decisions, specifically to find alternative coding idioms. Researchers have explored the energy impact of design patterns [10, 29, 40] and refactoring [35, 41]. However, they were not able to provide specific guidelines, perhaps because the energy footprints of these coding decisions were too small. Manotas and colleagues [30] achieved significant energy saving only by replacing Java Collections classes, but they were not able to explain what is contributing to the improvement. Instead, they followed a search-based software engineering approach to find the alternative that produces the most energy-efficient result.

Our work focuses on creating energy profiles of popular Java Collections classes in order to guide developers. Specifically, we created energy consumption profiles of commonly used API methods for variants of three Collections datatypes: List, Map, and Set. Then, we investigated the reasons behind the difference in energy profiles by considering memory usage and bytecode executed during an operation. Finally, we explored how the Collections instances are used in real applications and how the usage impacts the overall energy consumption. Using the per-method energy profiles as building blocks, a developer can estimate the energy impact of each Collections instance and choose a more efficient alternative, if available. An essential property is that the profiles respect the constraints developers are tied to, since developers choose a Collections class on purpose, e.g., a List instead of a Set or a Map. Hence, proposing a Set or a Map to swap a List is confusing as a guideline. This is different from an autotuning approach that aggressively swaps Collections classes based only on API match [30].

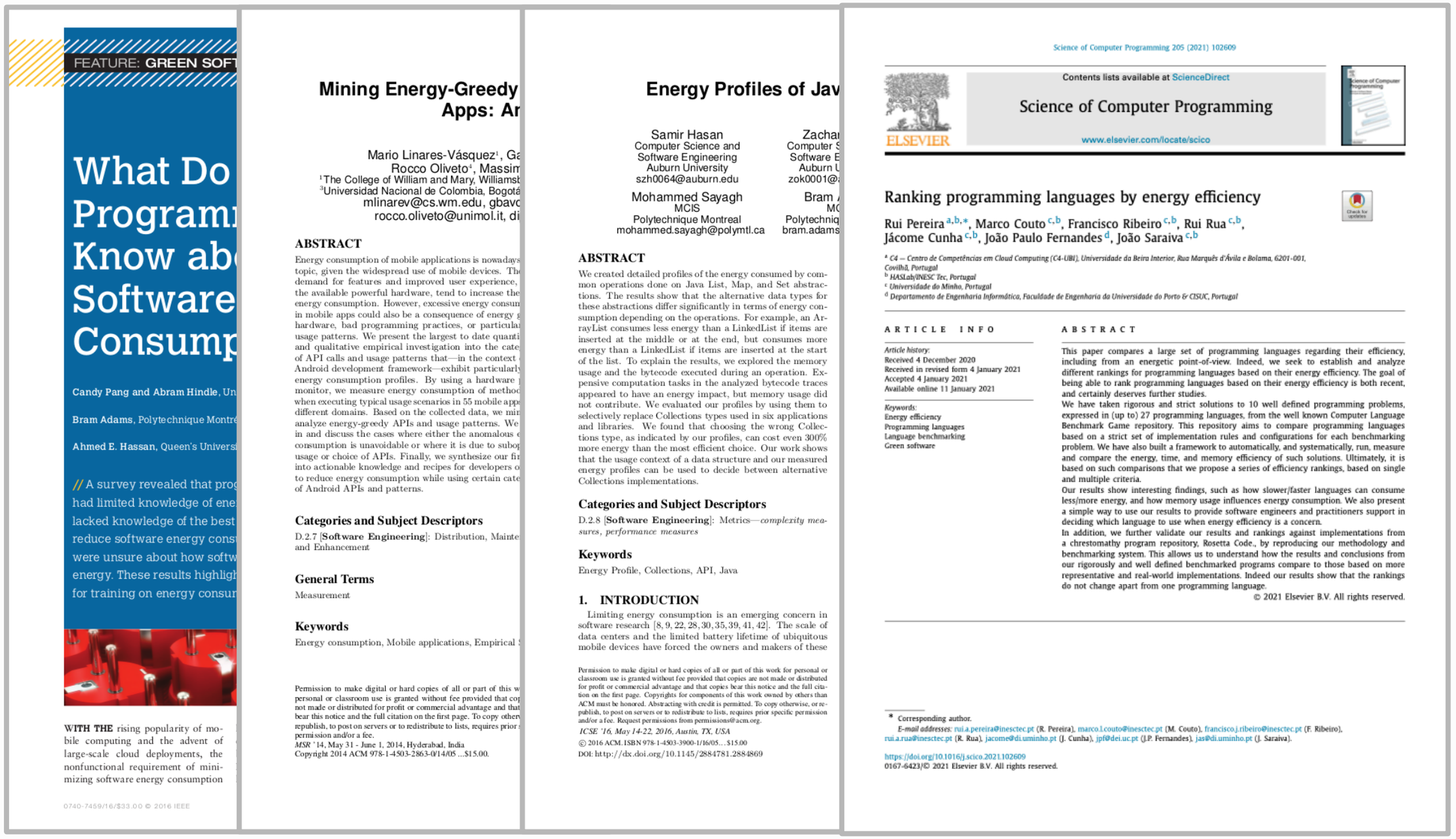
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICSE '16, May 14-22, 2016, Austin, TX, USA
© 2016 ACM. ISBN 978-1-4503-3900-1/16/05…$15.00
DOI: http://dx.doi.org/10.1145/2884781.2884869

The context in which data structures are used greatly affects energy consumption. Developers should carefully select from different Collections implementations.

https://dl.acm.org/doi/pdf/10.1145/2884781.2884869

Developers and Energy Consumption



FEATURE: GREEN SOFT

What Do Programi Know ab Software Consump

Candy Pang and Abram Hindle, Un

Bram Adams, Polytechnique Montre

Ahmed E. Hassan, Queen's Universi

// A survey revealed that prog had limited knowledge of ene lacked knowledge of the best reduce software energy cons were unsure about how softw energy. These results highligh for training on energy consur

WITH THE rising popularity of mobile computing and the advent of large-scale cloud deployments, the nonfunctional requirement of minimizing software energy consumption

0740-7459/16/$33.00 © 2016 IEEE

Mining Energy-Greedy Apps: Ar

Mario Linares-Vásquez[1], Ga Rocco Oliveto[4], Massim
[1]The College of William and Mary, Williamsb
[3]Universidad Nacional de Colombia, Bogotá
mlinarev@cs.wm.edu, gbavc rocco.oliveto@unimol.it, di

ABSTRACT

Energy consumption of mobile applications is nowadays topic, given the widespread use of mobile devices. The demand for features and improved user experience, the available powerful hardware, tend to increase the energy consumption. However, excessive energy consum in mobile apps could also be a consequence of energy g hardware, bad programming practices, or particular usage patterns. We present the largest to date quanti and qualitative empirical investigation into the cate of API calls and usage patterns that—in the context Android development framework—exhibit particularly energy consumption profiles. By using a hardware monitor, we measure energy consumption of method when executing typical usage scenarios in 55 mobile apps different domains. Based on the collected data, we min analyze energy-greedy APIs and usage patterns. We in and discuss the cases where either the anomalous consumption is unavoidable or where it is due to subop usage or choice of APIs. Finally, we synthesize our fi into actionable knowledge and recipes for developers o to reduce energy consumption while using certain cate of Android APIs and patterns.

Categories and Subject Descriptors

D.2.7 [Software Engineering]: Distribution, Mainte and Enhancement

General Terms

Measurement

Keywords

Energy consumption, Mobile applications, Empirical

Permission to make digital or hard copies of all or part of this w personal or classroom use is granted without fee provided that co not made or distributed for profit or commercial advantage and that bear this notice and the full citation on the first page. To copy other republish, to post on servers or to redistribute to lists, requires prior permission and/or a fee.
MSR '14, May 31 - June 1, 2014, Hyderabad, India
Copyright 2014 ACM 978-1-4503-2863-0/14/05 ...$15.00.

Energy Profiles of Jav

Samir Hasan
Computer Science and Software Engineering
Auburn University
szh0064@auburn.edu

Zachar
Computer S
Software E
Auburn U
zok0001@

Mohammed Sayagh
MCIS
Polytechnique Montreal
mohammed.sayagh@polymtl.ca

Bram
MC
Polytechniq
bram.adams

ABSTRACT

We created detailed profiles of the energy consumed by common operations done on Java List, Map, and Set abstractions. The results show that the alternative data types for these abstractions differ significantly in terms of energy consumption depending on the operations. For example, an ArrayList consumes less energy than a LinkedList if items are inserted at the middle or at the end, but consumes more energy than a LinkedList if items are inserted at the start of the list. To explain the results, we explored the memory usage and the bytecode executed during an operation. Expensive computation tasks in the analyzed bytecode traces appeared to have an energy impact, but memory usage did not contribute. We evaluated our profiles by using them to selectively replace Collections types used in six applications and libraries. We found that choosing the wrong Collections type, as indicated by our profiles, can cost even 300% more energy than the most efficient choice. Our work shows that the usage context of a data structure and our measured energy profiles can be used to decide between alternative Collections implementations.

Categories and Subject Descriptors

D.2.8 [Software Engineering]: Metrics—*complexity measures, performance measures*

Keywords

Energy Profile, Collections, API, Java

1. INTRODUCTION

Limiting energy consumption is an emerging concern in software research [8, 9, 22, 28, 30, 35, 39, 41, 42]. The scale of data centers and the limited battery lifetime of ubiquitous mobile devices have forced the owners and makers of these

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICSE '16, May 14-22, 2016, Austin, TX, USA
© 2016 ACM. ISBN 978-1-4503-3900-1/16/05…$15.00
DOI: http://dx.doi.org/10.1145/2884781.2884869

Science of Computer Programming 205 (2021) 102609

Contents lists available at ScienceDirect

Science of Computer Programming

www.elsevier.com/locate/scico

Ranking programming languages by energy efficiency

Rui Pereira[a,b,*], Marco Couto[c,b], Francisco Ribeiro[c,b], Rui Rua[c,b], Jácome Cunha[c,b], João Paulo Fernandes[d], João Saraiva[c,b]

[a] *C4 — Centro de Competências em Cloud Computing (C4-UBI), Universidade da Beira Interior, Rua Marquês d'Ávila e Bolama, 6201-001, Covilhã, Portugal*
[b] *HASLab/INESC Tec, Portugal*
[c] *Universidade do Minho, Portugal*
[d] *Departamento de Engenharia Informática, Faculdade de Engenharia da Universidade do Porto & CISUC, Portugal*

ARTICLE INFO

Article history:
Received 4 December 2020
Received in revised form 4 January 2021
Accepted 4 January 2021
Available online 11 January 2021

Keywords:
Energy efficiency
Programming languages
Language benchmarking
Green software

ABSTRACT

This paper compares a large set of programming languages regarding their efficiency, including from an energetic point-of-view. Indeed, we seek to establish and analyze different rankings for programming languages based on their energy efficiency. The goal of being able to rank programming languages based on their energy efficiency is both recent, and certainly deserves further studies.

We have taken rigorous and strict solutions to 10 well defined programming problems, expressed in (up to) 27 programming languages, from the well known Computer Language Benchmark Game repository. This repository aims to compare programming languages based on a strict set of implementation rules and configurations for each benchmarking problem. We have also built a framework to automatically, and systematically, run, measure and compare the energy, time, and memory efficiency of such solutions. Ultimately, it is based on such comparisons that we propose a series of efficiency rankings, based on single and multiple criteria.

Our results show interesting findings, such as how slower/faster languages can consume less/more energy, and how memory usage influences energy consumption. We also present a simple way to use our results to provide software engineers and practitioners support in deciding which language to use when energy efficiency is a concern.

In addition, we further validate our results and rankings against implementations from a chrestomathy program repository, Rosetta Code., by reproducing our methodology and benchmarking system. This allows us to understand how the results and conclusions from our rigorously and well defined benchmarked programs compare to those based on more representative and real-world implementations. Indeed our results show that the rankings do not change apart from one programming language.

© 2021 Elsevier B.V. All rights reserved.

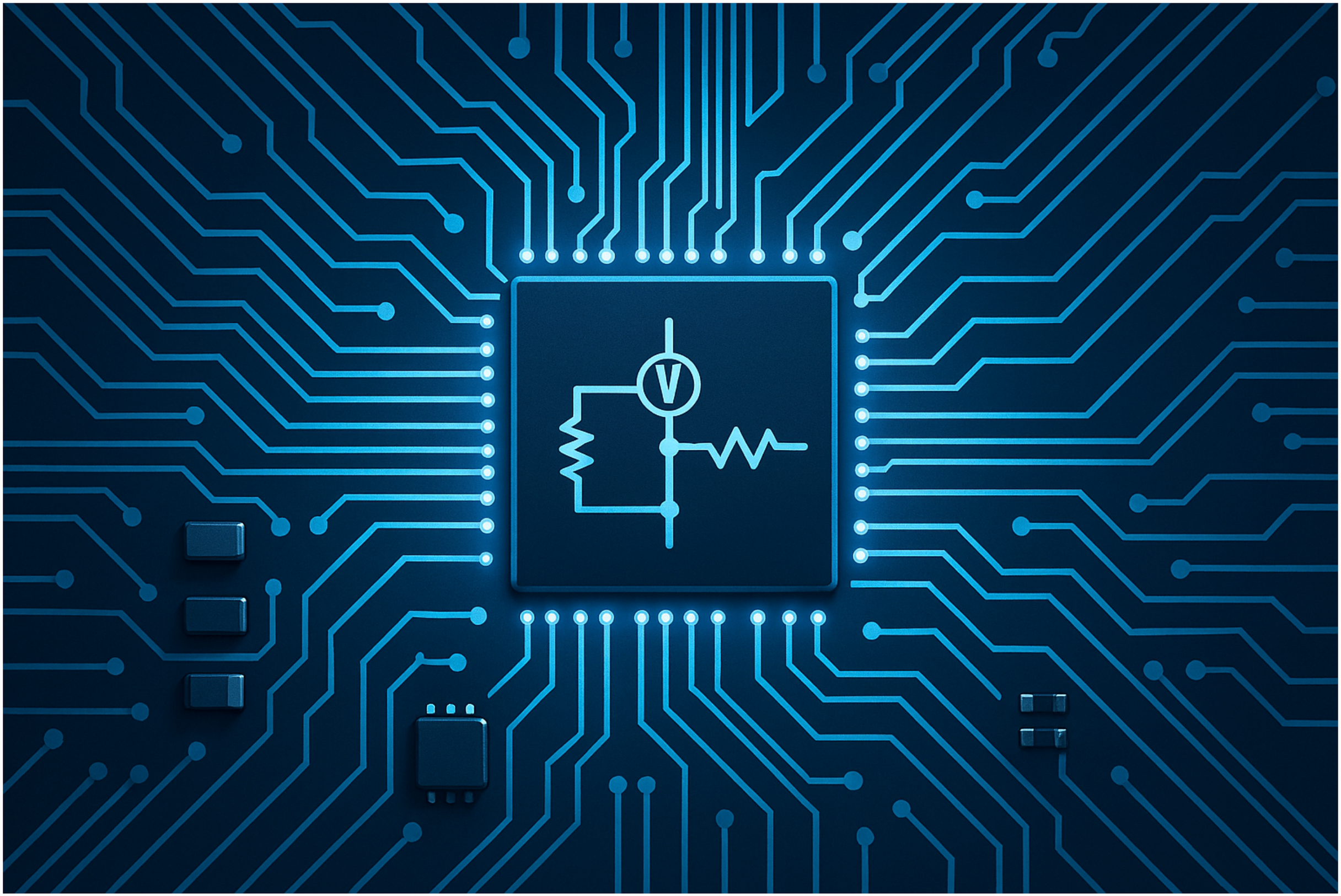
* Corresponding author.
E-mail addresses: rui.a.pereira@inesctec.pt (R. Pereira), marco.l.couto@inesctec.pt (M. Couto), francisco.j.ribeiro@inesctec.pt (F. Ribeiro), rui.a.rua@inesctec.pt (R. Rua), jacome@di.uminho.pt (J. Cunha), jpf@dei.uc.pt (J.P. Fernandes), jas@di.uminho.pt (J. Saraiva).

https://doi.org/10.1016/j.scico.2021.102609
0167-6423/© 2021 Elsevier B.V. All rights reserved.

Implementing effective coding practices, selecting efficient data structures, and choosing specific programming languages can greatly improve energy efficiency. Using energy monitoring tools and benchmarks offers valuable insights and increases developers' awareness of energy-related issues.

https://www.sciencedirect.com/science/article/pii/S0167642321000022

Let’s go back to the basics: Electrical Energy



Let’s go back to the basics: Electrical Energy

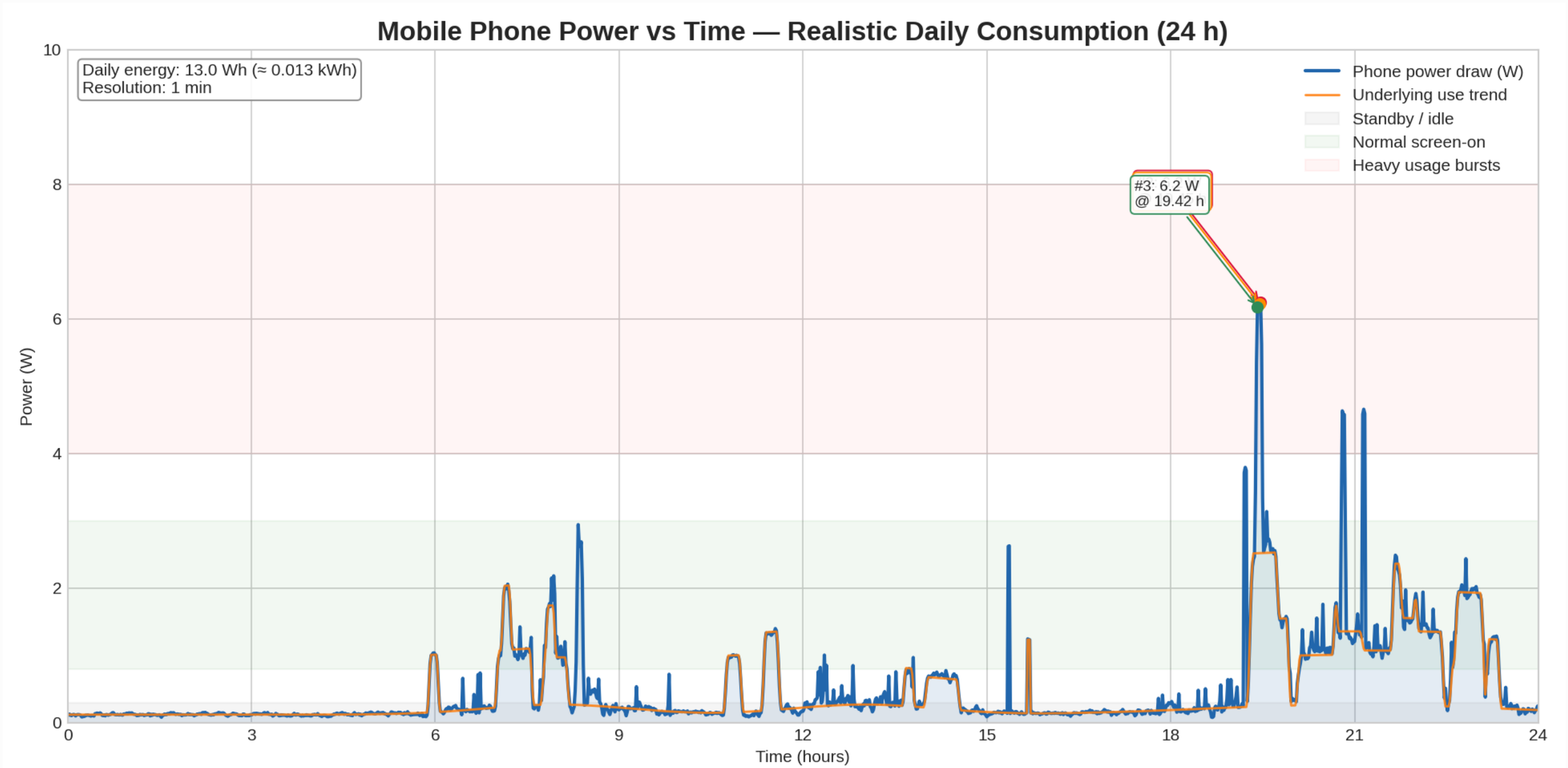
Electrical energy is the energy produced by the movement of electrons from one point to another in an electrical circuit, occurring when charged electrons flow.

Energy common measurement units

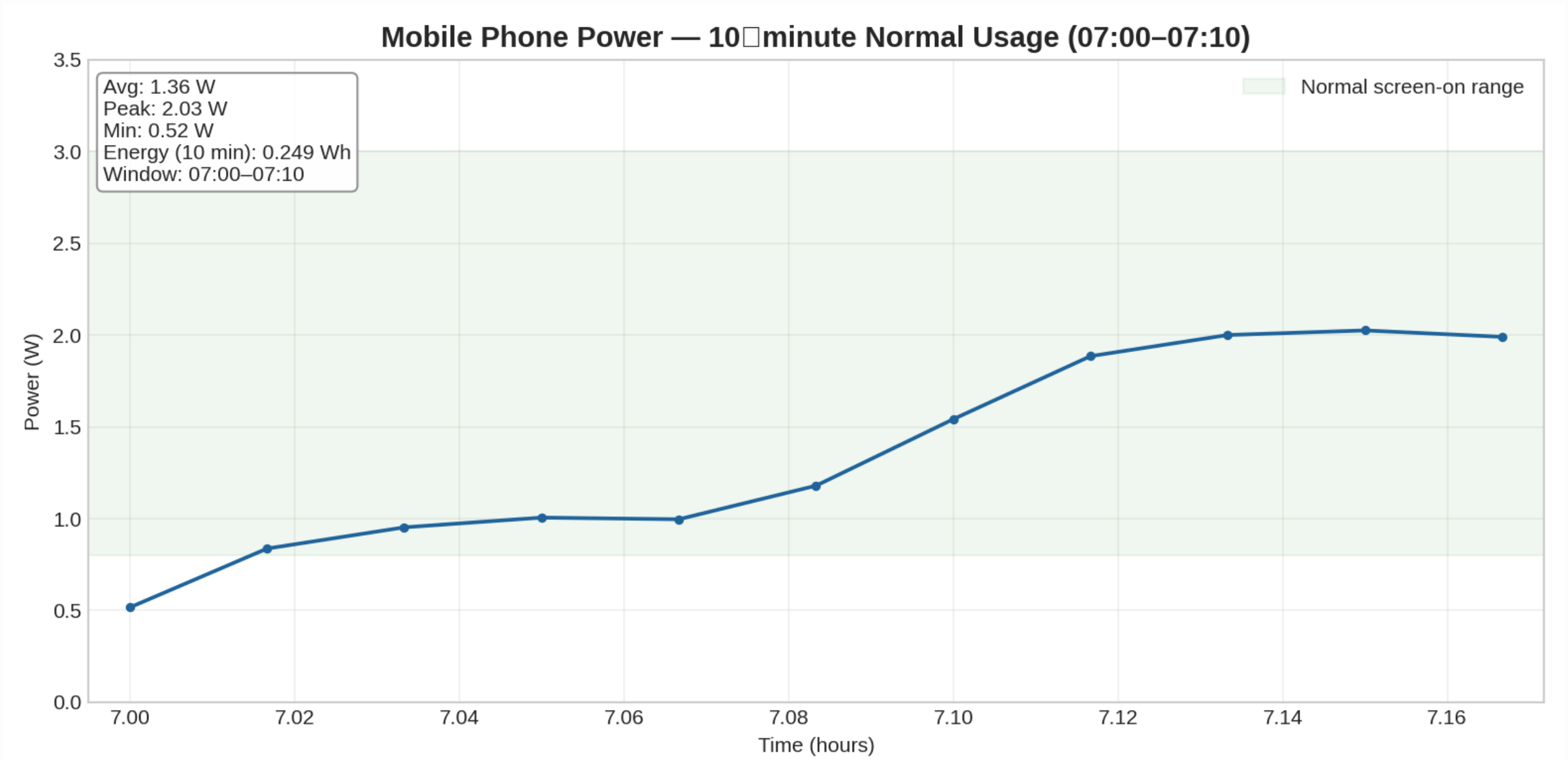
- Joule (J), the unit of energy or work in the Système international d'unités (SI). It is the total amount of energy consumed or produced by a device to perform a specific task or work.
- Watt (W), the standard SI unit of power, i.e., the energy transfer or work done per second. The rate at which a device consumes or generates energy; a higher watt rating indicates that more energy is used or produced per second.

$$1W = 1J/s$$

Let’s go back to the basics: Electrical Energy

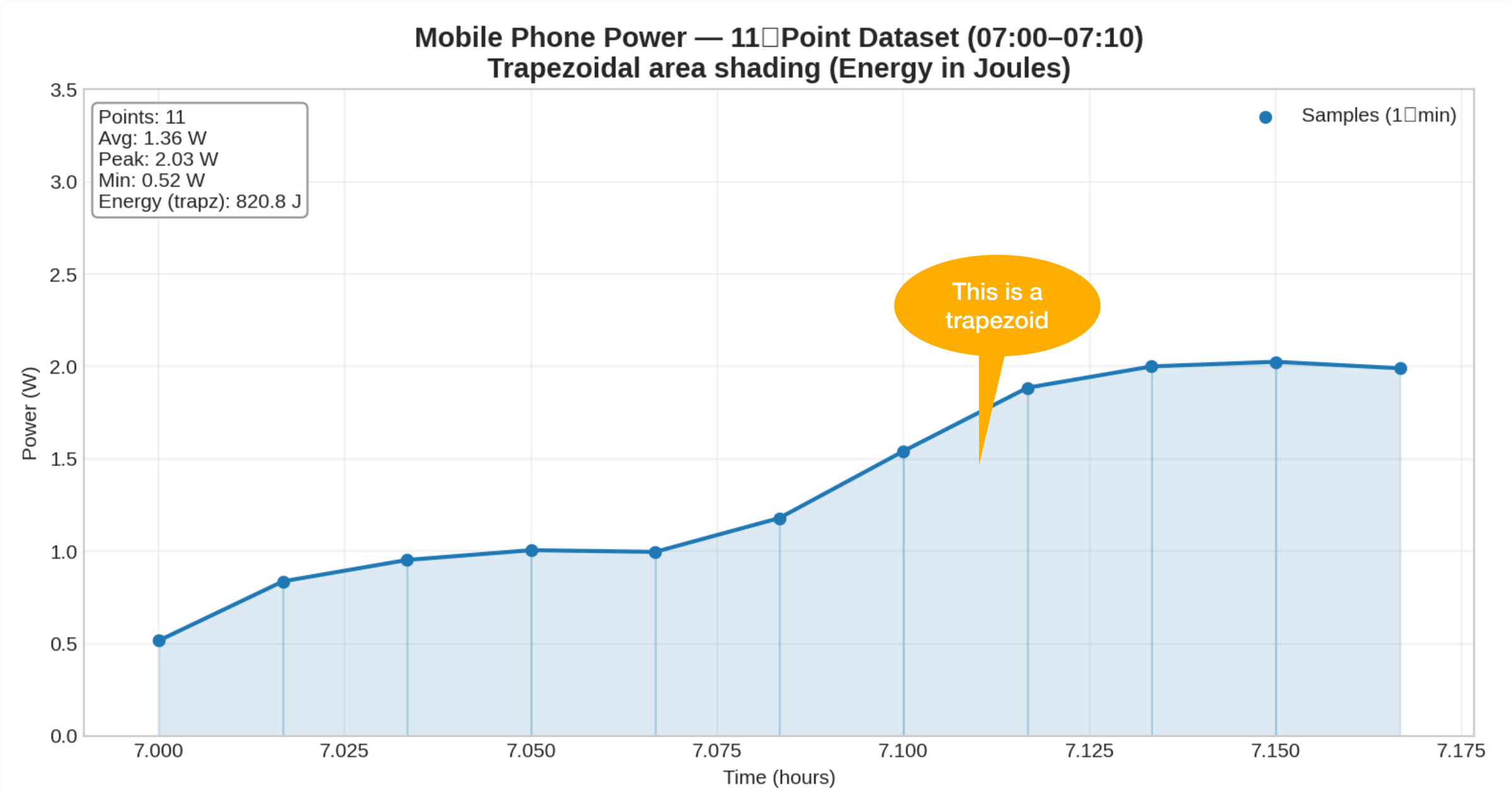


Let’s go back to the basics: Electrical Energy



$$Energy = \int P(t)dt$$

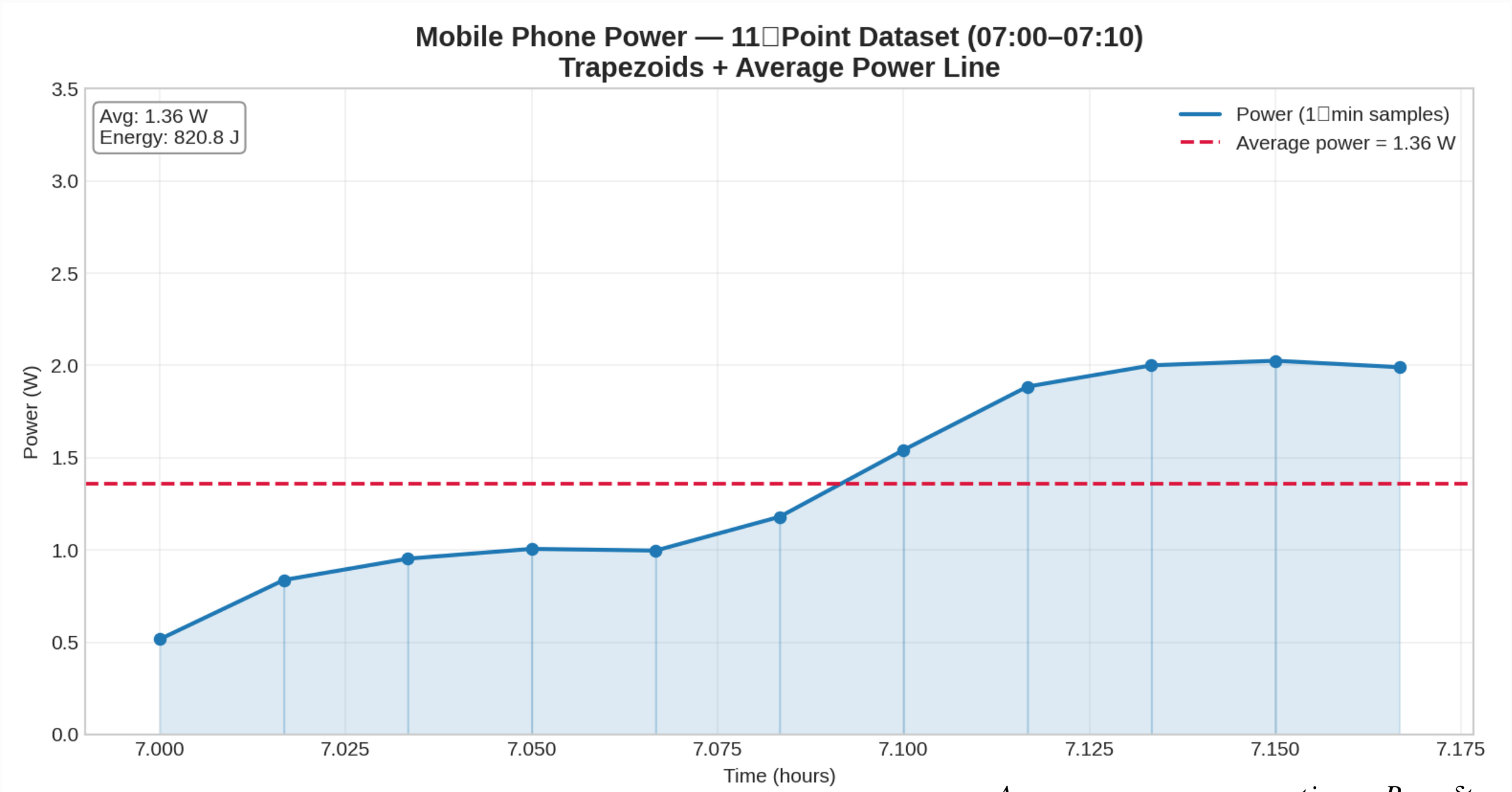
Let’s go back to the basics: Electrical Energy



The area of a single trapezoid can be computed as $A = \frac{a+b}{2}h$

The trapezoid rule $Energy\ consumption = \int_{t_0}^{t_n} P(t)dt \approx \frac{\Delta t}{2}[P(t0) + P(t1) + P(t2) + \ldots + P(t_{n-1}) + P(t_n)]$

Let’s go back to the basics: Electrical Energy



$$Average\ power\ consumption = P_{avg} \cdot \delta t$$

Gathering time data is crucial for estimating energy.

Average power consumption is useful for estimating energy in continuous use cases.

Energy consumption makes sense in one-time use cases.

Let's go back to the basics: Electrical Energy

To achieve lower energy consumption, you can run the software in low-power CPU mode.

This change may result in slower but more energy-efficient execution.

In reality, switching frequency takes time; therefore, we need a metric that penalizes slow runs.

$$EnergyDelayProduct = Energy \cdot t = P_{avg} \cdot \delta t^2$$

This metric is a good compromise between low consumption and fast runtime, as it gives more importance to the latter.

Let's go back to the basics: Electrical Energy



This battery above has a capacity of 3,600 mAh (milliampere-hours). What is a mAh?

The mAh is a unit of electric charge. It is equivalent to 3.6 C (Coulombs), which is the SI unit.

The energy that a battery can transfer is equal to $Energy = Voltage \cdot Charge$

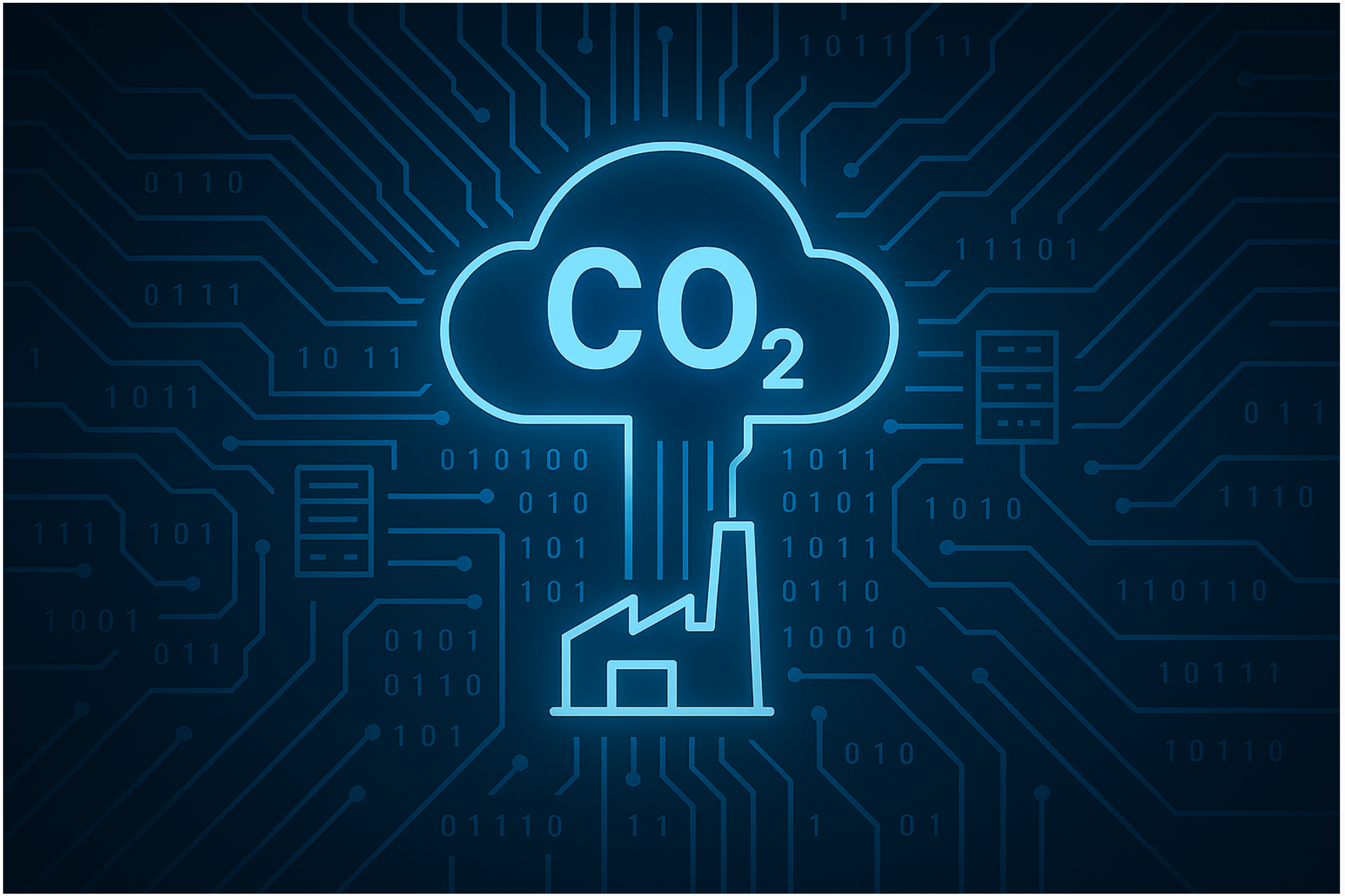
The above battery has a voltage of 14.8 V (Volts). Therefore, the transferable energy is $3{,}600mAh \cdot 14.8V = 53.28Wh = 53.28Wh$

Given that $1J = 1Wh \cdot 3{,}600$, the energy in J is $53{,}28Wh \cdot 3{,}600 \approx 191kJ$

We use charge units because the device voltage changes over time based on its needs.

Most devices use voltage to calculate their battery level percentage.

Carbon Emissions



Carbon Emissions

Greenhouse gases are a group of gases contributing to **global warming** and **climate change**.

The Kyoto Protocol, an environmental agreement adopted by many of the parties to the United Nations Framework Convention on Climate Change (UNFCCC) in 1997 to curb global warming, nowadays covers seven greenhouse gases:

- carbon dioxide (CO2)
- methane (CH4)
- nitrous oxide (N2O)
- hydrofluorocarbons (HFCs)
- perfluorocarbons (PFCs)
- sulphur hexafluoride (SF6)
- nitrogen trifluoride (NF3)

Converting them to carbon dioxide equivalents (CO2-eq) enables comparison and determination of their individual and total contributions to global warming.

https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Glossary:Greenhouse_gas_(GHG)

Carbon Emissions

Carbon dioxide equivalents are commonly expressed in millions of metric tonnes of carbon dioxide equivalents, abbreviated as MMTCDE.

The carbon dioxide equivalent for a gas is derived by multiplying the tonnes of the gas by the associated GWP:

$$MMTCDE = (million\ metric\ tonnes\ of\ a\ gas) * (GWP\ of\ the\ gas)$$

For example, the GWP for methane is 25, and for nitrous oxide is 298.

Emissions of 1 million metric tonnes of methane and nitrous oxide, respectively, are equivalent to emissions of 25 and 298 million metric tonnes of carbon dioxide.

https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Glossary:Carbon_dioxide_equivalent

Carbon Emissions



Energy Consumption vs Carbon Emissions

The emissions depend on each country's energy mix: https://app.electricitymaps.com/map/all_years/yearly

Lowering carbon emissions to zero is unfeasible due to marginal power plants.

A marginal power plant is the last one dispatched to satisfy an increase in electricity demand.

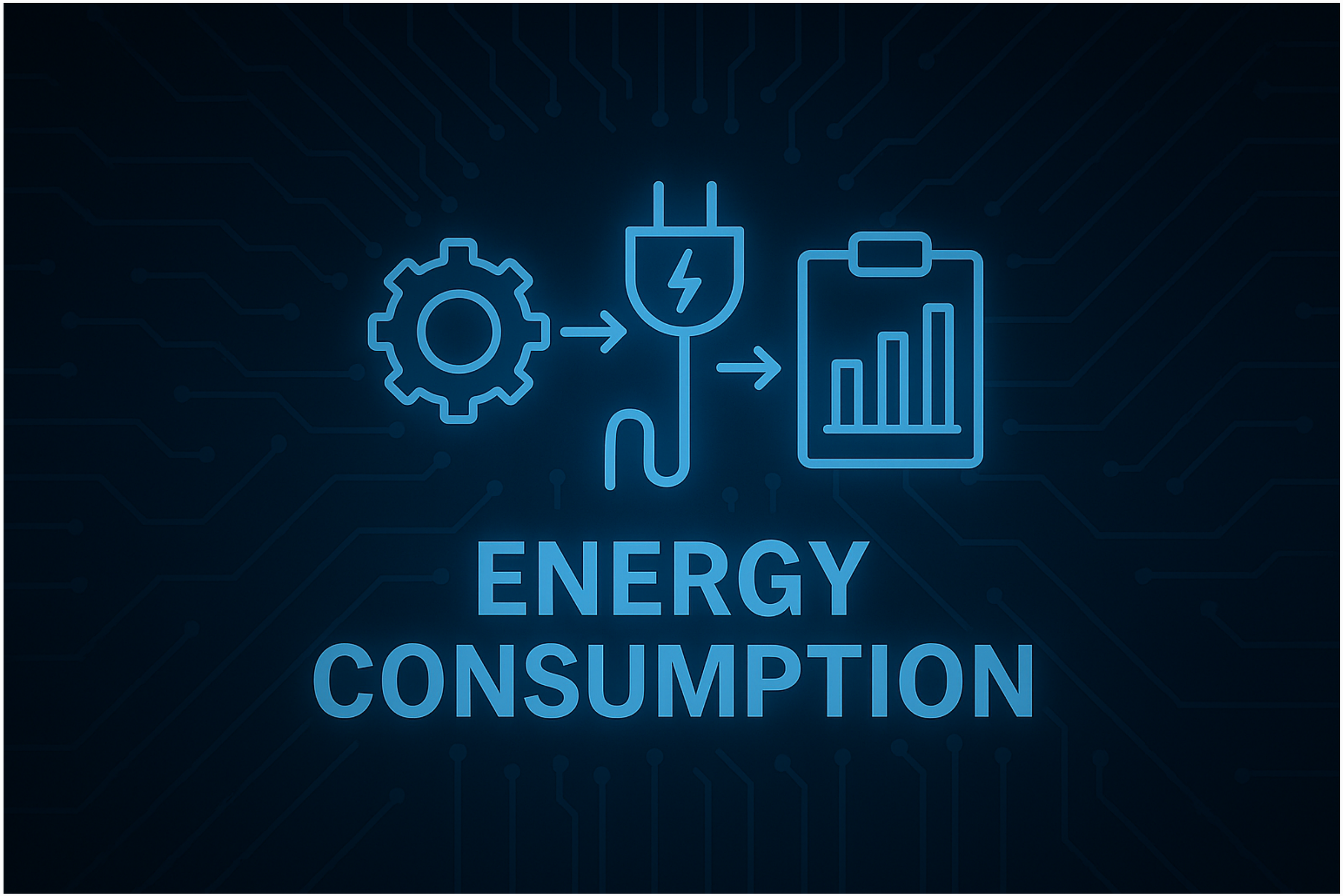
This plant sets the marginal cost and emissions for the extra electricity used, which is often generated by a fossil-fuel plant due to its ability to be dispatched and scaled up or down.

Should I calculate Energy Consumption or Carbon Emissions?

You should use energy consumption when estimating software consumption.

Carbon is more useful when estimating infrastructure consumption, e.g., a server farm.

How to Measure Energy Consumption?



How to Measure Energy Consumption?

Energy tests are not deterministic; running them multiple times yields different outputs.

Several factors must be minimized before using a measurement tool.

First, you need to reduce background activities:

- Close all running applications.
- Terminate all unnecessary background services.
- Turn off notifications.
- Turn off or unplug all unnecessary hardware, including Wi-Fi, if you do not need the internet.
- If you need internet, use a wired Ethernet connection.
- Switch off auto-brightness, keep the screen brightness fixed.

How to Measure Energy Consumption?

The environment is more important than in functional testing!

Hardware temperature impacts your results.

The higher the temperature, the greater the energy consumption.

Run some tests without measuring the temperature to warm up the machine.

Conduct all tests in the same room with a stable temperature.

Alternatively, collect data about the temperature to exclude measurements taken under different environmental conditions.

Since energy tests are not deterministic, you need to run them multiple times.

Make sure to pause between runs to prevent the results of one run from affecting the next.

Automatically shuffling clusters of tests can also be helpful.

sesa lab
SOFTWARE ENGINEERING
SALERNO

sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

Sustainable Software Engineering - Energy Metrics and the Measurement Procedure



Slides courtesy of Dario Di Nucci | 2026 - 2027